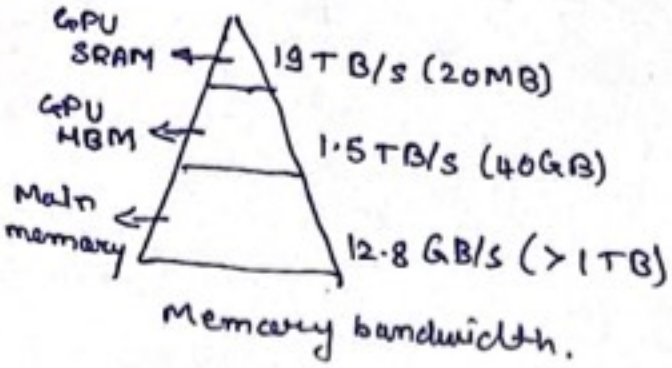
Flash Attention

This provides speedups, Training and inferrence, speeds up attention mechanism by optimising what values are loaded and moved between GPU's shared memory (SRAM) and high bandwidth memory (HBM)

Transformers have problem of quadratic scaling

1 token → 1 parameter 2 token → 4 parameters.

Flash attention provides 3x speedup on GPT 2, 15% end to end wall clock speed



Standard attention

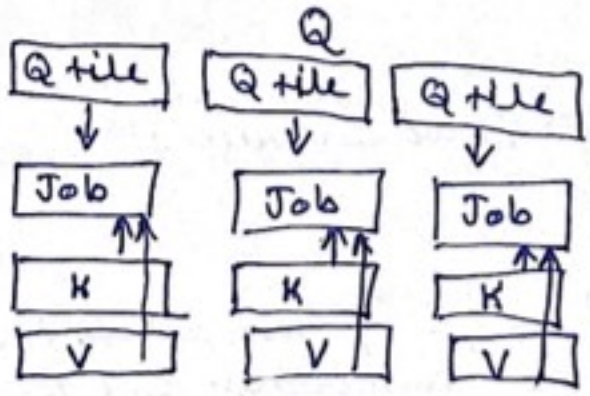
QKV -> HBM

1. Load Q, K from HBM
   compute $QK^T = S$
2. Read S, $P = \text{softmax}(S)$
   P -> HBM
3. Load P and V from HBM
   $O = PV$
4. Return O

Each token compares itself to every key turns those similarities into probability distribution with softmax and then averages value vectors using those probabilities as weights.

How Flash Attention changes this process (computation)

- It splits Q into query tiles, splits K, V into key-value tiles
- Loads one block at a time into SRAM
- compute partial score blocks
- update softmax and output incrementally
- Never stores full score P in HBM



Standard attention computes all pairwise scores first then softmax then multiply by V

Flash attention. compute a small score block then fold it into softmax/output then discard the block and move onto next block

Online Softmax (class notes)

How can we calculate softmax without seeing the whole row at once?

Trick is to maintain for each query row, two running statistics while scanning blocks:

- the running row maximum m_i ($f(x)$)
- the running normalized denominator l_i ($l(x)$)

$$\text{Softmax} = \frac{m_i}{l_i} = \frac{f(x)}{l(x)} = [e^{x_i - m(x)} \ldots\ldots e^{x_B - m(x)}]$$

$$l(x) = \sum_i f(x)_i$$

$$\text{softmax} = \frac{f(x)}{l(x)}$$

e.g. $x^{(1)}, x^{(2)} \in \mathbb{R}^B$

$$x = [x^{(1)}\ x^{(2)}] \in \mathbb{R}^{2B}$$

$$m(x) = m([x^{(1)}\ x^{(2)}]) = \max[(m(x_1)), (m(x_2))]$$

$$f(x) = e^{m(x_1) - m(x)}$$

$$f(x_1) = e^{m(x_1) - m(x)} \cdot l(x_1) + e^{m(x_2) - m(x)} \cdot l(x)$$

$$\text{softmax} = \frac{f(x)}{l(x)}$$

Simplified:

Imagine you are standing at a door and 100 people are walking through the door one by one. you need to find tallest person and we can only remember height of only one person at a time.

you dont need to gather all people in a room at once (HBM). we can keep a record of a person's height passing through the door in our head (SRAM) we update max value when we find someone taller.

Safe softmax

we must address a numerical overflow. If we compute e^{100}, 16 bit GPU register will overflow

To fix this we use safe softmax

$$\text{softmax}(x_i) = \frac{e^{x_i - m}}{\sum_{j=1}^{N} e^{x_j - m}}$$

, e^{-m} factors out and cancelled in numerator and denominator. but largest factor now is $e^0 = 1$ which makes the range of values 0 to 1. This makes softmax stable

Flash attention continued

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

Unrolling this into GPU operations

1) Matrix multiplication 1: $S = \frac{QK^T}{\sqrt{d}}$

$d \times N$ matrix results in S, an $N \times N$ matrix

2) Softmax: $P = \text{softmax}(S)$

Shape is still $N \times N$ matrix

3) Matrix Multiplication 2: $O = PV$

An $N \times N$ matrix $\times$ $N \times d$ matrix $=$ $N \times d$ matrix

Lets assume we split our row vector into two blocks $x^{(1)}$ and $x^{(2)}$

Online Softmax

Local max: $m^{(1)} = \max(x^{(1)})$ // max of block 1

Local sum: $d^{(1)} = \sum e^{x^{(1)} - m^{(1)}}$ // safe/stable denominator 0 to 1

Step 2: Process Block 2 & Update Globally

$m^{(2)} = \max(m^{(1)}, \max(x^{(2)}))$ // max of both blocks

<u>Update running denominator</u>

Scale old sum down by difference in maximums

$d^{(2)} = d^{(1)} \cdot e^{m^{(1)} - m^{(2)}} + \sum e^{x^{(2)} - m^{(2)}}$ // scaled down denominator

This is online softmax.

This calculation allows to calculate max and sum in a single pass

Forward pass - Flash attention v1

- Allocate empty matrix O $(N \times d)$ and two running statistic vectors m and l $(N \times 1)$ in HBM
- outer loop: Load a block of queries Q_i in SRAM
- inner loop: - Load K_j and V_j into SRAM - calculate $S_{local} = Q_i K_j^T$
  - update running max (m) and running denominator l
  - Scale previous o/p O_i and add new local o/p $P_{local} V_j$

- Flush: once inner loop finishes divide O_i by final l.
  Write completely accurate block back to slow HBM

Memory access is reduced from $O(N^2)$ to $O(N)$

Backward Pass & recomputation Trick

During backward pass, flash attention recomputes P locally on SRAM

1. Load a block of Q and K. Compute S_{local}
2. We saved global max (m) and sum (l) during forward pass
   So we skip expensive softmax calculations
3. Calculate $P_{local} = \frac{e^{S_{local} - m}}{l}$

4. Compute gradients, add them back to HBM
   Delete P_{local} again